Investigation of the supplied captures

The organizer's capture set is normalized with the same path the application uses (`src/capture/pipeline.py`) and read back through the capture contract, so these numbers describe the shipped files and the shipped code, not a fixture.

1. Research captures

The request rule (`examples/thermostand_request_v2.json`) and the response rule (`examples/thermostand_response_v2.json`) are applied to every research capture. `not_applicable` counts the messages the other direction carries, so a clean capture reports only `matched`.

Requests:

capture	matched	mismatched	incomplete	not_applicable
01_configuration.pcap	14	0	0	14
02_variations.pcapng	15	0	0	15
03_combined_messages.pcap	8	0	0	8
04_new_observations.pcapng	8	0	0	8

Responses:

capture	matched	mismatched	incomplete	not_applicable
01_configuration.pcap	14	0	0	14
02_variations.pcapng	15	0	0	15
03_combined_messages.pcap	8	0	0	8
04_new_observations.pcapng	8	0	0	8

No message is mismatched, incomplete or ambiguous in any research capture. The two exceptions in the response stream of the variations capture are error replies (`value_out_of_range`, `malformed_body`): they carry a one-byte body, so the configuration fields after the status are correctly out of the message and the fields that belong to a successful reply are not applied.

2. Refinement from a counterexample

The first reading (`examples/thermostand_request_v1.json`) assumes the length field counts the payload only and that a message starts at the first byte of the length field. Applied to the configuration capture:

rule	matched	mismatched	incomplete
v1 (payload-only length)	0	2	4
v2 (length includes the four-byte header)	14	0	0

The counterexamples v1 produces, each tied to its bytes:

status	field	reason
incomplete	message_counter	field_beyond_message
mismatched	magic	value 1 not in expected [42588]
incomplete	magic	payload_truncated
incomplete	message_counter	field_beyond_message
mismatched	magic	value 1 not in expected [42588]
incomplete	magic	payload_truncated

The fix is a four-byte header bias on the length framing, which v2 carries as `length_bias`. The counterexamples are the evidence: the first message under v1 is framed four bytes short, so the magic is read at the wrong offset and the counter lands outside the message.

3. Diagnostic captures

Each diagnostic capture is built to exercise one failure mode. The diagnostics below are what the reassembler actually reports.

capture	exercises	diagnostics reported
01_retransmission.pcap	retransmission	none
02_reordered.pcapng	out-of-order delivery	none
03_missing_segment.pcap	a missing segment	gap
04_snaplen_truncated.pcapng	a truncated snapshot	gap
05_conflicting_overlap.pcap	a conflicting overlap	ambiguity
06_missing_beginning.pcapng	a connection with no start	none
07_checksum_offload.pcap	checksum offloading	none
08_bad_checksum.pcapng	a wrong checksum	none

Checksum verification is off by default, so `07_checksum_offload.pcap` reports `none`. With `--verify-checksums` it reports `checksum_offload`.

4. Journal correlation

The action journal holds 109 entries. The decoded requests correlate as follows: 14 of 14 messages correlated, 95 journal entries unmatched

The correlation ties each request message to the operator action it performs and confirms the decoded parameters against the recorded values, so a field name is not merely a plausible label.

5. Where the set comes from

The composition and the data origin are recorded by the organizer in `tests/corpus/real/dataset_manifest.json`: the expected session and action counts per research capture, and the statement that the application bytes come from a real client/simulator TCP socket exchange while the Ethernet/IP/TCP framing, segmentation and clock are generated. The pipeline reproduces the recorded session counts (2, 1, 1, 1) exactly, and the eight diagnostic captures each carry one capture or integrity condition. The full analysis is in `docs/REAL_CORPUS_ANALYSIS.md`.